

Errors and Control Flow — A Readable Companion to the Elixir Docs

Chapter 2 · checked against Elixir 1.20 · examples run on Elixir 1.14 and 1.18 · Sep 26, 2026

1. Why Elixir has two ways to fail

Elixir separates two kinds of failure that many languages mix together:

- **Expected failures** are part of the domain: the file might not exist, the user might not be found, the input might not parse. They are *values*, usually `{:error, reason}`, and the caller decides what to do.
- **Unexpected failures** are bugs or broken assumptions: a key that must be there isn't, a function gets data it was never designed for. They *raise*, the process crashes, and a supervisor restarts it from a clean state (Erlang book, chapter 1).

Most error-handling mistakes in Elixir come from blurring the two. Rescuing an exception that signals a bug hides it. Raising for something routine makes a normal condition crash a process. Returning `nil` for either makes both invisible.

2. The building blocks

Tool	Use for
<code>{:ok, value} / {:error, reason}</code>	Expected outcomes the caller must handle
<code>foo!</code> (bang) variant	The same operation, raising instead of returning an error: for when failure means a bug
<code>case</code>	Branching on one result
<code>with</code>	A sequence of steps that each might fail
<code>raise / rescue</code>	Unexpected conditions; rescuing only at clear boundaries
<code>throw / catch</code>	Rare: escaping from code you don't control
<code>exit</code>	Process-level failure; handled by links and supervisors, not <code>try</code>

The convention the docs describe: `File.read/1` returns `{:ok, binary} | {:error, reason}`, and `File.read!/1` returns the binary or raises. Use the plain version when failure is a normal possibility, and the bang when it would mean something is broken.

with in one example

```
with {:ok, user} <- Accounts.fetch(id),  
     {:ok, order} <- Orders.create(user, params),
```

```
      :ok          <- Mailer.confirm(order) do
    {:ok, order}
  end
```

Each `<-` must match or the whole `with` stops. The docs: “while `=` raises in case of not matches, `<-` will simply abort the `with` chain and return the non-matched value.”

3. How it actually works

A failed with returns whatever didn't match

Without `else`, the value that failed to match is returned as-is. If every step returns `{:error, reason}` on failure, that is exactly what you want. If one step returns something else (`false`, `nil`, `:error`), that value leaks to the caller.

else must be complete

With an `else` block, a non-matching value goes to `else`. If no `else` clause matches it, `with` raises `WithClauseError`. A partial `else` turns an unhandled error shape into a crash.

after is a soft guarantee

`try ... after` runs its cleanup when the block finishes or raises. But if the process is killed, for example by its supervisor during shutdown or by a linked crash, `after` does not run. The docs call these “soft guarantees.” Resources that must be released need an owner process whose death releases them (Erlang book, chapter 4 on ETS ownership and chapter 11 on ports).

Exceptions have a cost

Raising and rescuing captures a stacktrace and unwinds the stack. That's fine for rare events, but expensive and hard to follow as ordinary control flow. The design anti-patterns guide names this directly: *exceptions for control flow*.

4. Plausible but wrong

4.1 Parsing by catching exceptions

```
def parse_port(s) do
  try do
    {:ok, String.to_integer(s)}
  rescue
    ArgumentError -> {:error, :invalid}
  end
end
```

This is the *exceptions for control flow* anti-pattern: a routine case, bad input, goes through raise and rescue. Fix: use the non-raising API, and check the whole result, because `Integer.parse("12x")` returns `{12, "x"}`, not an error:

```
def parse_port(s) do
  case Integer.parse(s) do
    {n, ""} when n in 1..65535 -> {:ok, n}
    _ -> {:error, :invalid}
  end
end
```

4.2 A with step that doesn't return an error tuple

```
def checkout(id) do
  with {:ok, user} <- Accounts.fetch(id),
       true <- user.active do
    {:ok, place_order(user)}
  end
end
```

For an inactive user this returns `false`. The caller matches on `{:ok, _}` and `{:error, _}` and crashes with `CaseClauseError`, or worse, treats `false` as “no error”. Fix: make every step return the same shape:

```
defp ensure_active(%{active: true}), do: :ok
defp ensure_active(_), do: {:error, :inactive}
```

4.3 A partial else

```
with {:ok, user} <- fetch(id),
     true <- user.active do
  {:ok, user}
else
  {:error, reason} -> {:error, reason}
end
```

`false` goes to `else`, matches no clause, and raises `WithClauseError`. Adding `else` made the function crash instead of returning an odd value. The anti-pattern guide also warns against large `else` blocks that try to decode which step failed. Fix: normalise errors in small helpers (as in 4.2), so the `with` needs no `else` at all.

4.4 Rescuing everything

```
def handle_call({:import, rows}, _from, state) do
  result =
    try do
      {:ok, Importer.run(rows, state.schema)}
    rescue
```

```

      _ -> {:error, :import_failed}
    end
    {:reply, result, state}
  end
end

```

A `KeyError` from a bug in `Importer` now looks exactly like bad input. There is no crash report and no stacktrace, the supervisor never restarts anything, and the bug can live for months. Fix: rescue only the specific exceptions you expect and can act on, return `{:error, reason}` from `Importer` for expected failures, and let everything else crash.

4.5 A bang on user input inside a server

```

def handle_call({:load, path}, _from, state) do
  {:reply, :ok, %{state | data: File.read!(path)}}
end

```

A mistyped path from a user is an expected failure, but `File.read!/1` raises and crashes the server, losing its state and failing every other caller's request in flight. Fix: `File.read/1` and reply `{:error, reason}`. Keep bangs for things that must exist, like a bundled asset at startup.

4.6 An option that changes the return type

```

def parse(input, opts \\ []) do
  data = do_parse(input)
  if opts[:raw], do: data, else: {:ok, to_struct(data)}
end

```

Callers can't tell from the call site what shape comes back, and one missed option turns into a pattern-match error somewhere else. The design anti-patterns guide calls this *alternative return types*. Fix: two functions with fixed return types, `parse/1` and `parse_raw/1`.

4.7 Relying on after for cleanup

```

def with_lock(key, fun) do
  :ok = Locks.acquire(key)
  try do
    fun.()
  after
    Locks.release(key)
  end
end

```

If the calling process is killed while `fun` runs, for example a `Task` that times out and is shut down, `after` never runs and the lock is never released. Fix: have the lock server monitor the holder and release the lock when it gets `:DOWN`, so the lock's lifetime is tied to the process itself.

5. Review checklist and sources

- ❑ Expected failures return `{:ok, _}` / `{:error, _}`; bang functions are used only where failure means a bug
- ❑ No `try/rescue` used to parse, validate or branch on routine conditions
- ❑ Every step in a `with` returns `{:ok, _}` / `:ok` or `{:error, _}`; helpers normalise other shapes
- ❑ `with ... else` blocks are small and complete, or absent
- ❑ `rescue` names specific exceptions; no bare `rescue _`
- ❑ Options never change a function's return type
- ❑ Resources that must be released are tied to a process's lifetime, not only to `after`

Sources

- `try`, `catch`, and `rescue`: <https://hexdocs.pm/elixir/try-catch-and-rescue.html>
- `with` (Kernel.SpecialForms): <https://hexdocs.pm/elixir/Kernel.SpecialForms.html#with/1>
- Design-related anti-patterns: <https://hexdocs.pm/elixir/design-anti-patterns.html>
- Code-related anti-patterns: <https://hexdocs.pm/elixir/code-anti-patterns.html>